INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Programmation du Drone et du Jeu

S. Delprat

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Prérequis

Pour ce projet intégratif, il vous faut:

- 1 PC avec Python 3.10

  https://www.python.org/downloads/release/python-3100/ (fichier au bas de la page)

- Matlab R2023a et suivantes

  Ne fonctionne pas en R2022

- Les toolboxes suivantes: iterativeDisplay & flappyDrone

  => disponibles sur Moodle

  => depuis Matlab, cliquer sur les fichiers ".mtblx" pour les installer

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Architecture logicielle

Piloter le drone sans jouer

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

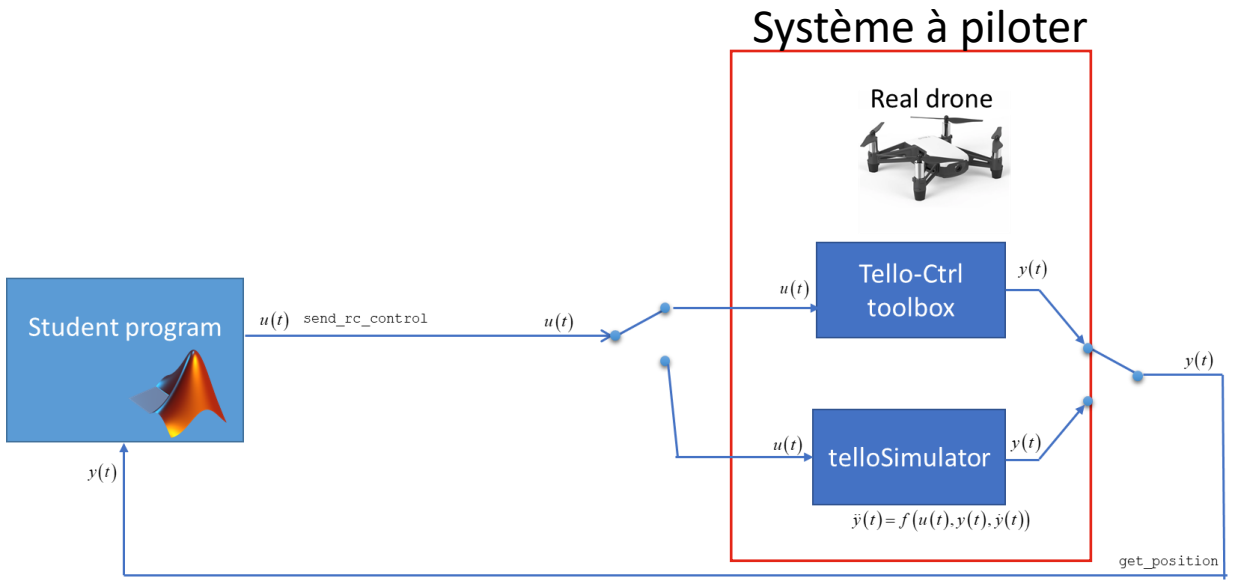
- Vous avez la possibilité de piloter soit le drone réel soit le simulateur

=> Le code ne change quasiment pas

- On envoie la commande [-100,100] au drone avec `send_rc_control`.

- Le drone renvoie sa hauteur avec `get_position`.

Ce slide ne présente que l'architecture de la commande, il y a d'autres étapes pour faire voler le drone



Architecture logicielle

Piloter le drone sans jouer

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

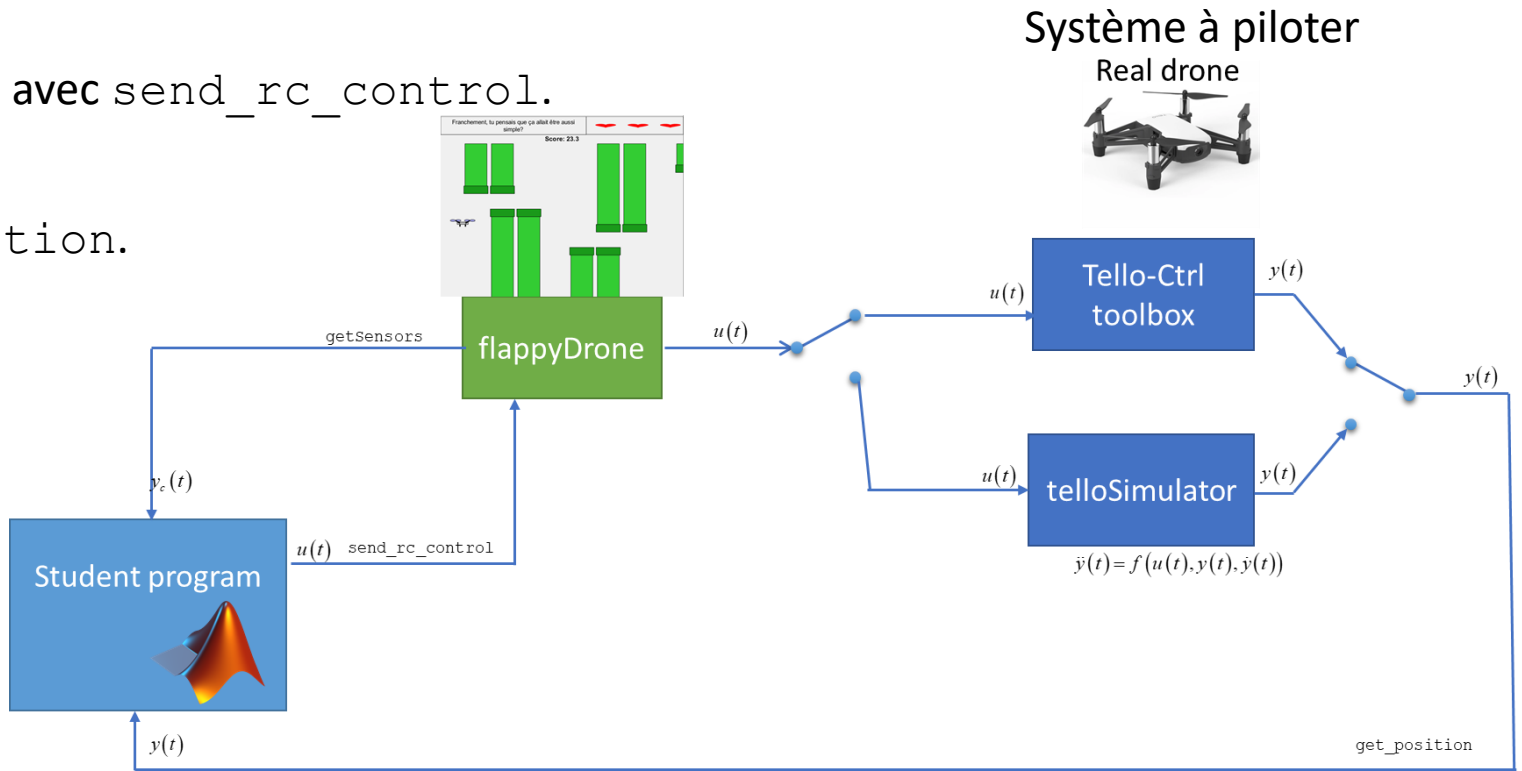
- Vous avez la possibilité de piloter soit le drone réel soit le simulateur

=> Le code ne change quasiment pas

- L'objet `flappyDrone` gère le jeu.
  Il reçoit la commande à envoyer au drone via `send_rc_control`
  Il renvoie la consigne via `getSetpoint`

- On envoie la commande [-100,100] au drone avec `send_rc_control`.

- Le drone renvoie sa hauteur avec `get_position`.



Ce slide ne présente que l'architecture de la commande, il y a d'autres étapes pour faire voler le drone

Pilotage du drone

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Créer un objet tello

```
setupDroneEnv;
tello=getTelloDrone();
```

Initialisation & installation si nécessaire

Crée l'objet tello

- Etablir la connexion (nécessite que le PC soit connecté au Wifi du drone)

```
tello.connect();
```

Fréquence de fonctionnement : 5 Hz

- Décoller (prend entre 4 et 8 secondes)

```
tello.takeoff();
```

- Envoyer une position de Joystick : entière entre -100 et 100.

```
FD.send_rc_control(int8(LR), int8(FB), int8(UD),int8(Yaw));
```

Left-right

Front-back

Up-down

Rotation

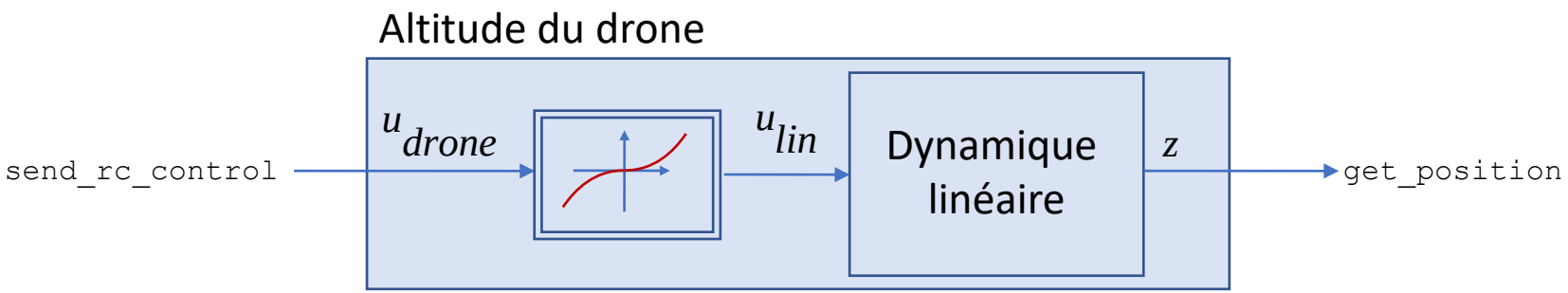
Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	**Pilotage du drone**	**8**
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Pilotage du drone

Le drone est conçu pour être piloté par un humain. Pour que le pilotage soit plus facile, une fonction non linéaire est appliquée à l'entrée reçue.

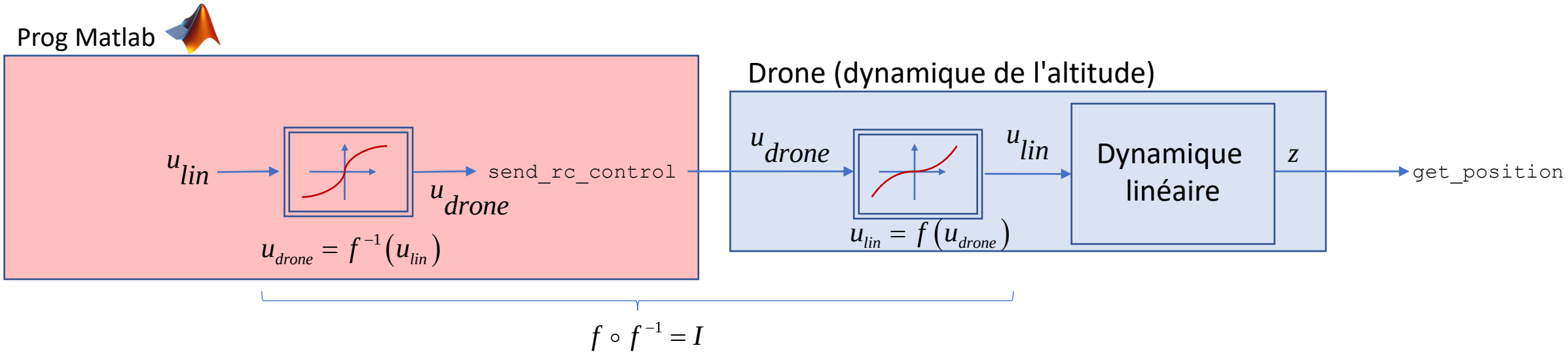
On note $u_{lin} = f\left(u_{drone}\right)$ cette non linéarité.



Cette non-linéarité peu être problématique pour l'automaticien débutant car cela rend le système non-linéaire.
Heureusement, il est possible de "contrer" cette non-linéarité.

Pilotage du drone

La fonction non-linéaire étant connue est inversible, vos programme n'ont simplement qu'à déterminer la commande u_{lin} pour la dynamique linéaire puis la "convertir" en $u_{drone:}$



Les non-linéarités "s'annulent"

Les deux fonctions f et f^{-1} sont modélisées par des cartographies (i.e. les valeurs sont obtenues en utilisant des interpolations linéaires)

Pilotage du drone

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

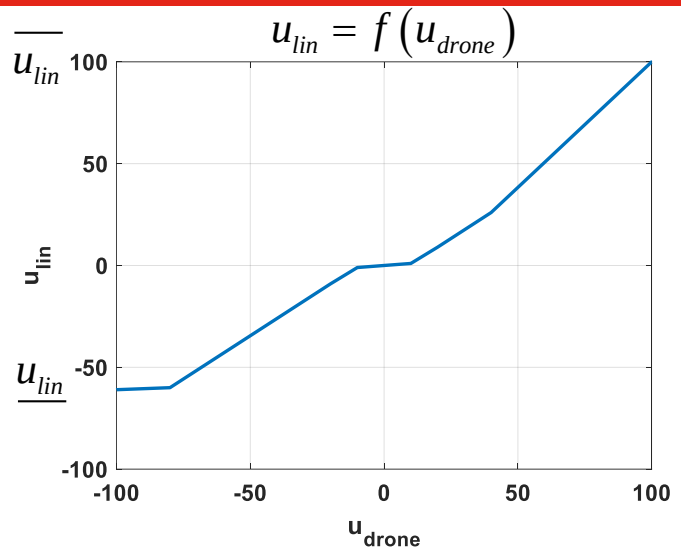
Vous devez donc:

1) Charger le fichier "`nonlinearity.mat`"
2) Calculer une commande `uLin` pour piloter la dynamique linéaire
3) Saturer la commande:

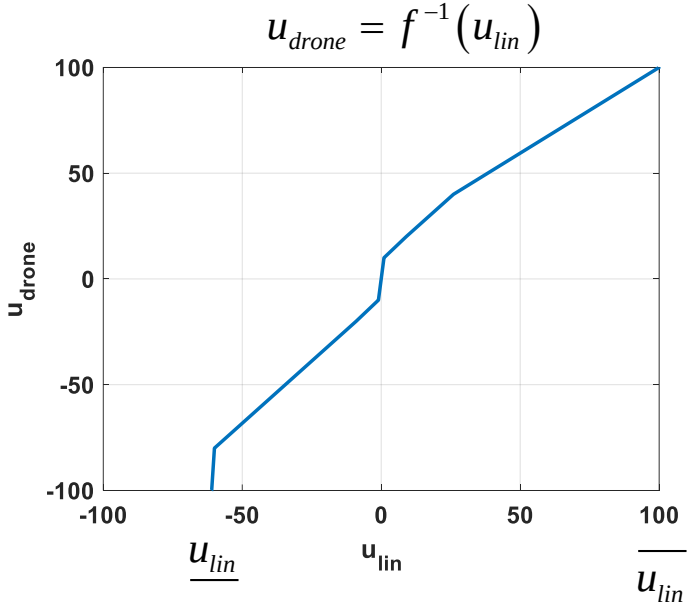
```
uLin=min(nonlienarity.uLinMax,max(nonlinearity.uLinMin,uLin));
```

1) Calculer la commande uDrone à envoyer au drone

```
uDrone=interp1(nonlinearity.uLin, nonlinearity.uDrone,uLin);
```



Avec ce code, on peut espérer que la dynamique entre `uLin` et l'altitude mesurée `z` soit linéaire



Contenu du fichier nonlinearity :

```
nonlinearity =

  struct with fields:

     uDrone: [-200 -100 -80 -40 -20 -10 0 10 20 40 100 200]
       uLin: [-110 -61 -60 -26 -9 -1 0 1 9 26 100 110]
    uLinMax: 100
    uLinMin: -61
```

Récupérer les données

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

L'objet Tello fonctionne en Python. Les données sont récupérées dans Matlab avec un format spécifique (objet Python).
Il est obligatoire de les convertir en "double" avant toute utilisée dans Matlab

- Récupérer l'état de la batterie

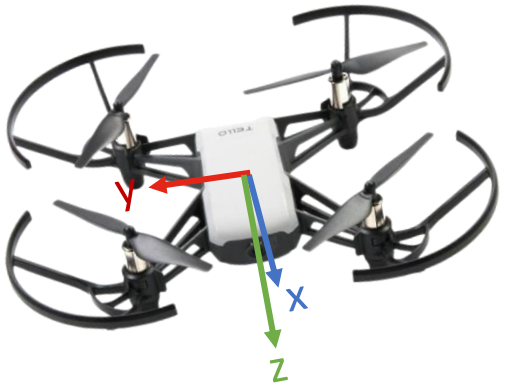
```
batt=double(tello.get_battery());
```

Conversion au format double de Matlab

- Récupérer la position

```
pos=double(tello.get_position());
altitude=-pos(4);
```

pos = [x,y,z,**tof**]



x,y,z : valeurs de position estimées, peu fiable
tof : valeur mesurée par le capteur, très fiable

Attention le repère du drone a l'axe Z orienté vers bas

Fin de mission

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Atterrir

```
tello.land()
```

- Nettoyage de fin de programme

```
tello.quit();
terminate(pyenv);
```

Si vous terminez pas "proprement" votre programme, il est possible que l'exécution suivante fasse planter Windows!!

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Sécurité logicielle

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Lorsqu'on développe le code d'un système automatisée, il faut qu'un être humain puisse toujours avoir la main sur le système

En cas de dérive du drone, un être humain doit toujours pouvoir:

- Appliquer de petites corrections manuellement
- Arrêter le programme

C'est à vous d'adapter vos programme de sorte à ce que ces 2 exigences soient satisfaite.

NB: Le drone étant autonome, il n'est pas possible de l'arrêter avec un arrêt d'urgence physique.
=> c'est pour cela qu'il vole derrière un filet.

Sécurité logicielle

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Le système proposé consiste à :

- Utiliser une figure (existante ou crée pour l'occasion) pour capturer les évènements clavier

```
figure;
startManualControl;   % Capture les évènements clavier
```

+ "up" - "down" : avancer/reculer
+ "gauche" - "droite" : gauche/droite
+ "Shift" – "Ctrl": tourner horaire/antiho
+ "5" ou "return":

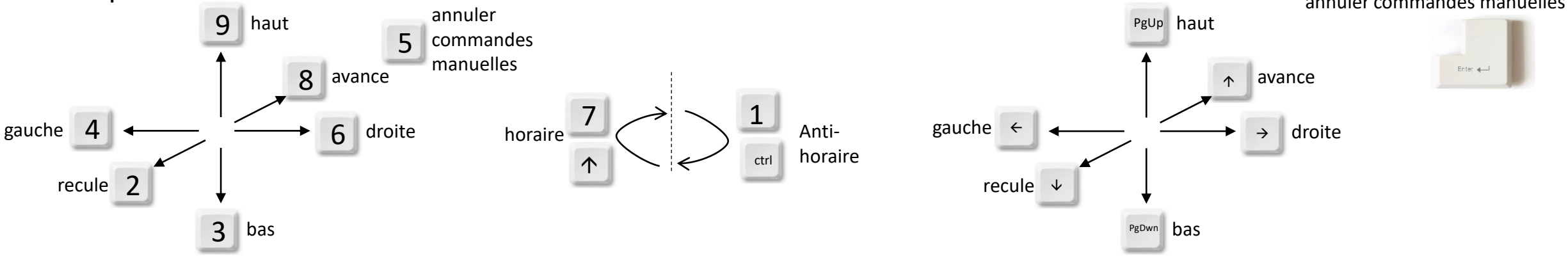
Grâce à cela, à chaque appuis sur une touche du clavier, on récupère la touche dans `keypressed` et une structure `manu`:

```
keypressed=evalin('base','globalKeypressed');
manu=evalin('base','manu');
```

La structure `manu` est la correction manuelle à envoyer à la commande:

```
tello.send_rc_control(int8(0+manu.LR), int8(0+manu.FB), int8(Uz+manu.UD),int8(0+manu.Yaw));
```

Corrections possible au clavier:



Sécurité logicielle

Il est donc attendu que si la touche pressée est "escape" ou "q" (à vous de choisir), le programme s'arrête.

Une fois le drone posé, vous devez arrêter le mécanisme de capture des touches

```
stopManualControl
```

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	**Le simulateur et le jeu**	**18**
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Le Simulateur

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Le simulateur fonctionne de manière identique au drone. Cependant il ne supporte que quelques fonctions.

```
tello=telloSimulator(useRealTime);
```

Si useRealTime = true : le simulateur fonctionne en temps réel est met à jour la position toutes les 0,2 secondes.
Si useRealTime = false : le simulateur suppose qu'entre 2 appels à `send_rc_control`, il s'est écoulé exactement 0,2 secondes même si ce n'est pas vrai
=> permet de simuler la dynamique du drone plus vite que le temps réel.

Le simulateur ne simule que le mouvement vertical. Toutes les autres dynamiques sont ignorées.

La syntaxe est identique à celle du drone réel.

Le Jeu

L'environnement du jeu est géré par un objet "`flappyDrone`"

```
FD=flappyDrone(tello,
               viewMode=1, ...
               animFrequency=10,
               useRealTime=true,
               Ts=Ts);
```

- `tello` : Simulateur ou drone réel
- `viewMode=1` : 1 : vue "jeu", 2 vue du niveau dans son ensemble
- `animFrequency=10` : Fréquence en Hz de rafraîchissement
- `useRealTime=true` : True : temps réel, False : le jeu suppose qu'il se passe exactement Ts secondes entre 2 appels de send_rc_control
- `Ts=Ts` : Période d'échantillonnage pour le mode non temps réel

Une fois que le drone a décollé, vous pouvez lancer le niveau souhaité:

```
FD.start(levelNo);
```

Le Jeu

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Pendant le jeu, vous devez envoyer vos commande au jeu et non pas au drone:

```
FD.send_rc_control(int8(0+manu.LR), int8(0+manu.FB), int8(Uz+manu.UD),int8(0+manu.Yaw));
```

Une fois le jeu arrêté, quelque soit la raison : niveau terminé, vous avez perdu, arrêt d'urgence, etc:

```
FD.stop;
```

Pendant le jeu, vous pouvez connaître l'état du jeu par

```
FD. isRunning
```

Renvoie True tant que le jeu est en cours (fin du niveau non atteinte et il vous reste des vies)

A la fin du jeu, vous pouvez enregistrer votre partie

```
FD. saveGame(filename)
```

Le Jeu

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Liste des propriétés:

FD.victory	True si la partie est gagnée
FD. remainingTime	Temps restant pour atteindre la fin du niveau
FD.totalDuration	Durée du niveau
FD.isRunning	True si une partie est en cours
FD.levelEndReached	True si la fin du niveau a été atteinte
FD.remainingLife	Nombre de vies restantes
FD.nbLevels	Nombre de niveaux disponible dans le fichier chargé
FD.score	Votre score
FD.droneX	Position du drone = temps écoulé
FD.droneY	Hauteur du drone

Le Jeu

Liste des fonctions:

Info=FD.getLevelInfo(noLevel)

Renvoie une structure avec les informations suivantes sur le niveau `noLevel`.

hasWind, title, predictionAllowed,
gateWidth,maxWind,nbLifes,duration,nbGates

Yc=FD.getSetpoint() ou Yc= FD. getSetpoint(x)

`FD.getSetpoint()` renvoie la consigne courante.
`FD.getSetpoint(x)` renvoie la consigne aux positions x demandées (x est un scalaire ou un tableau)
=> disponible que si le niveau a sa propriété "`predictionAllowed`" a true cf. `getLevelInfo`.

FD.displayALevel(noLevel)

Affiche une figure avec l'intégralité du niveau.

FD. start(noLevel)

Démarre le niveau `noLevel`.

Le Jeu

Liste des fonctions:

```
FD.stop()
```

Termine le niveau en cours

```
FD. saveGame(fileName)
```

Enregistre les informations d'une partie terminée dans le fichier spécifié.
L'extension du fichier est obligatoirement `.fdg`.

```
FD.loadGame(fileName)
```

Affiche le niveau sauvegardé avec la trace du drone

```
FD. displayAnimationStatistics()
```

Affiche des information sur les performances graphiques.
Pour debogage.

Le Jeu

Liste des fonctions:

```
FD.send_rc_control(left_right, forward_backward, up_down, yaw)
```

Envoie les commande au drone

Pendant le jeu, vous ne pouvez pas envoyer les commandes au drone directement.
Vous devez les envoyer au jeu qui les transmettra au drone.

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	**Gérer le temps avec Matlab**	**26**
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Le temps

Matlab permet une gestion "basique" du temps.

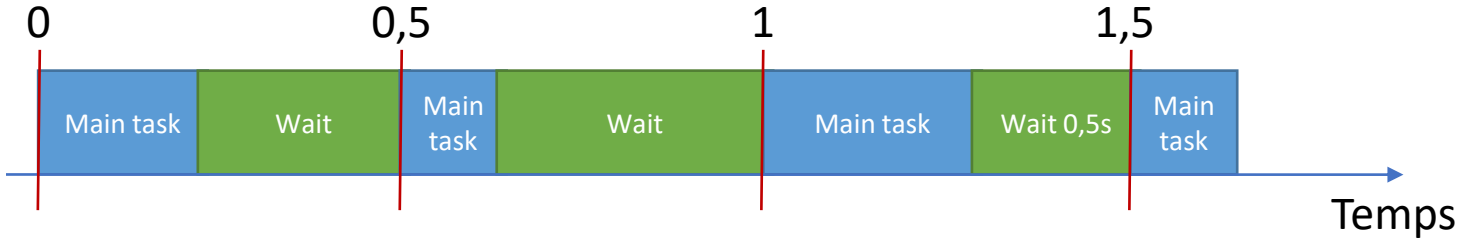
```
% Record current time
tStart=tic;

% Do something

% Get duration
duration = toc(tStart);
```

Objectif à atteindre:

Exécuter une tâche, de durée éventuellement variable, avec une période fixe

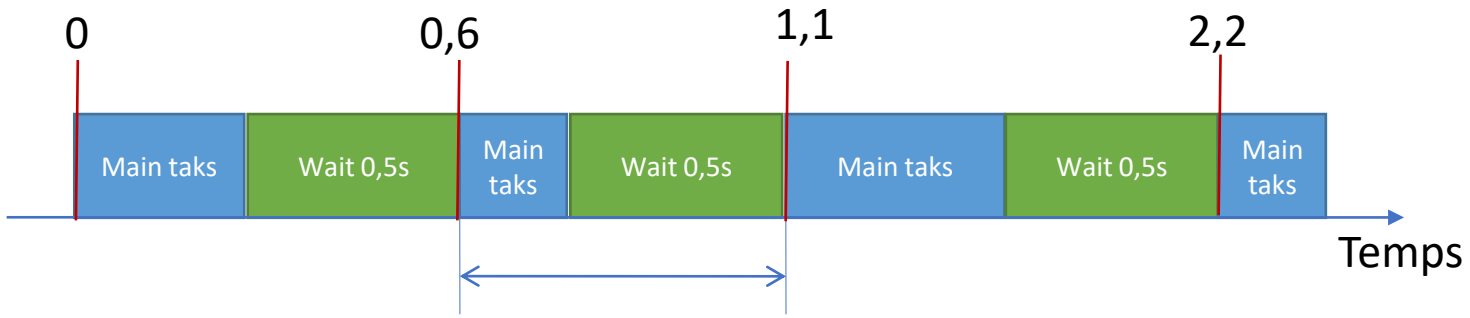


Exemple d'une mauvaise gestion du temps

Il faut faire attention à la structure du code. Par exemple, le code ci-dessous ne permet pas d'exécuter la tâche principale toute les 0,5 secondes: la durée d'exécution de la tâche (inconnue a priori) n'a pas été prise en compte.

```
% Record current time
tStart=tic;
for i=1:10
    % Main task
    fprintf('%i\n',i)
    f=0;
    for j=1:1e7
        f=f+sin(rand());
    end
    pause(.5); % Wait for 0,5 seconds
end
duration = toc(tStart);
fprintf('Duration : %.2f',duration)
```

Tâche à réaliser toutes les 0,5s



Période effective > 0,5s
Fluctue si la durée d'éxécution de la tâche est variable

Résultat

Duration : 7.23>>

Sommaire

I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Générer un nom de fichier unique

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Lorsque vous faites un essais, vous allez probablement vouloir garder une trace de ce qui a été fait, sans effacer d'éventuels enregistrements précédent.

Pour cela, `uniqueFileName` permet de générer automatiquement un nom de fichier unique:

essai_001.mat, essai_002.mat, etc.

Syntaxe : `[fileNameWithoutExt, fullfileName] = uniqueFileName(Folder, basename, ext)`

Paramètres d'entrée:

- Folder : répertoire où seront enregistrées les données
- basename : nom de fichier (sans le numéro qui sera ajouté automatiquement)
- Ext : nom de l'extension utilisée. Optionel, valeur par défaut ".mat"

Paramètres de sortie:

- `fileNameWithoutExt` : nom de fichier sans l'extension
- `fullfileName` : nom de fichier avec l'extension

La fonction lastFileName permet de récupérer le nom du dernier fichier enregistré.

Syntaxe : `[fileNameWithoutExt, fullfileName] = lastFileName(folder,basename,ext)`

=> reprend les mêmes paramètres entrées et sortie de uniqueFileName

Générer un BIP

Pendant un essai, vous allez surement regarder le drone et pas l'écran du PC.

Avoir un avertissement sonore peut alors s'avérer utile. La fonction `bip` génère un bip.

Syntaxe : `bip(freq,duration)`

Paramètres d'entrée, tous optionels:

- `freq` : Fréquence du BIP, par défaut 2kHz.
- `duration` : durée du bip en secondes. Par défaut 0,1s.

Sommaire

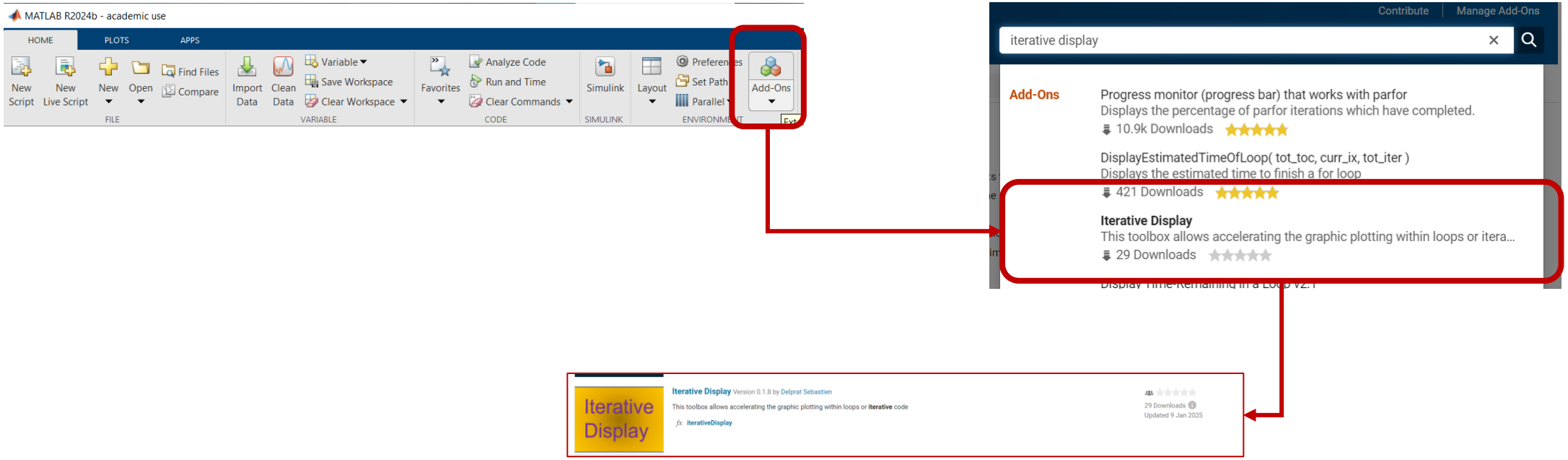
I.	Prérequis	2
II.	Architecture Logicielle	4
III.	Pilotage du drone	8
IV.	Sécurité logicielle	14
V.	Le simulateur et le jeu	18
VI.	Gérer le temps avec Matlab	26
VII.	Fonctions additionnelles	29
VIII.	Affichage graphiques accélérés	32

Graphiques accélérés avec iterativeDisplay

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Lorsqu'on met au point une loi de commande, il est important de surveiller le comportement de la boucle fermée.
Une bonne pratique est de visualiser la consigne et la sortie sur un graphe et la commande sur un autre.

Cependant l'affichage de graphique prend "beaucoup" de temps pour être mis à jour à chaque itération de la boucle de commande.
Afin de remédier à ce problème, vous pouvez utiliser la toolbox iterativeDisplay.



Graphiques accélérés avec iterativeDisplay

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

L'utilisation de iterativeDisplay accélère de 4 à 100 fois les affichages graphiques

Le principe d'utilisation est simple:

- Avant la boucle principale, créez un objet avec `id=iterativeDisplay();`
- Dans la boucle principale
  - En début d'itération appelez `id.newIteration` (ou `id.finalIteration;` à la dernière itération)
  - Rajouter "id." devant les commandes graphiques

```
id.figure;
id.plot(x,y);
```

⚠ Limitations ⚠

Les instructions séparées par des espaces : hold on, grid on, etc. ne sont pas supportées
A la place, vous devez utiliser la forme "fonction": id.hold('on'), id.grid('on'), etc.

Graphiques accélérés avec iterativeDisplay

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Exemple

```
clearvars
close all
clc

id=iterativeDisplay();
n=100; % Number of iterations
t=0:0.1:10;

% Create figure
id.figure;

for i=1:n
    % Add this piece of code at the begining of your main loop
    if i<n
        % Start a new iteration
        id.newIteration;
    else
        % Last iteration needs to be indicated
        id.finalIteration;
    end

    id.plot(t,sin(t+2*pi*(i-1)/n)); % Plot some data
    id.grid('on');
    id.xlabel('Time');
    id.ylabel('data');
    id.title("Iter "+i);
    drawnow; % Force display update
end
```

L'appel a id.finalIteration permet d'afficher à la fin des éléments graphiques qui ralentissent beaucoup l'affichage (ex les legendes).
=> Si vous n'appelez par id.finalIteration, ce n'est pas grave.